



MotoTrack

BLOC 2 — RNCP 39583 · EXPERT EN DÉVELOPPEMENT LOGICIEL

Concevoir et développer des applications logicielles

Dossier professionnel de certification appliqué au projet **MotoTrack** :
environnements & CI/CD, conception d'un prototype sécurisé,
développement testé, recette fonctionnelle et documentation technique.

Next.js 14 · App Router

TypeScript 5

PostgreSQL 15 · Prisma 5

Jest · Playwright

GitHub Actions · Vercel · Supabase

RGAA · OWASP Top 10 · ISO/IEC 25010

Auteur

Luca Straputicari

Établissement

Ynov Campus — Mastère Expert en développement logiciel

Nature du projet

Initiative personnelle issue d'une analyse de marché

Version applicative

1.0.0

Dépôt

github.com/<org>/MotoTrack

Sommaire

1 Introduction et cadrage

- Présentation de MotoTrack et du contexte
- Objectifs fonctionnels et périmètre de l'application
- Stack technique et justification des choix
- Méthodologie et organisation du travail

2 Préparation des environnements & CI/CD

- Environnement de développement local (outillage, conteneurisation Docker)
- Environnements de test et de déploiement (dev / staging / prod)
- Outils de suivi de performance et de qualité (KPI, linting, monitoring)
- Pipeline d'intégration continue (GitHub Actions : build, tests, lint)
- Gestion du code source (Git Flow, branches, PR & review)

3 Conception & architecture

- Architecture logicielle (App Router, couches, modèle de données Prisma)
- Choix ergonomiques et cibles (web / mobile, PWA)
- Sécurité pensée dès la conception (auth JWT, validation SIV)
- Design patterns, best practices et prototype

4 Développement des fonctionnalités

- Implémentation des fonctionnalités clés (Auth, Mon Garage, Carnet d'entretien, Tableau de bord)
- Sécurisation du code (OWASP, validation, gestion des secrets)
- Accessibilité (RGAA / WCAG)
- Évolutivité et maintenabilité
- Harnais de tests unitaires et code coverage
- Déploiement continu et progressif + vérification auprès des utilisateurs

5 Recette des fonctionnalités

- Cahier de recettes (scénarios de tests, résultats attendus)
- Exécution de la recette et détection des anomalies
- Analyse des anomalies et régressions
- Plan de correction des bogues (processus, priorisation, suivi)

6 Documentation technique

- Stratégie et outils de documentation
- Synthèse des manuels (réalisation / utilisation / déploiement)
- Traçabilité et support aux évolutions futures

7 Conclusion

- Bilan technique et fonctionnel
- Difficultés rencontrées et solutions
- Perspectives d'évolution de MotoTrack
- Bilan personnel / montée en compétences

8 Annexes

▸ Matrice de traçabilité des compétences

Compétence	Chapitre	Preuve principale dans le dépôt
C2.1.1 — Environnements & suivi qualité/perf	2	<code>docker-compose.yml</code> , <code>next.config.js</code> , <code>/api/health</code> , <code>/api/monitoring/vitals</code>

C2.1.2 — Intégration continue & sources	2	<code>.github/workflows/ci.yml</code> , Pull Requests
C2.2.1  — Prototype & architecture	3	<code>src/app</code> , <code>prisma/schema.prisma</code> , <code>src/lib</code> , <code>middleware.ts</code>
C2.2.2  — Tests unitaires & couverture	4	<code>tests/unit</code> (146 tests), <code>jest.config.js</code>
C2.2.3  — Sécurité & accessibilité	4	<code>rateLimit.ts</code> , <code>next.config.js</code> , <code>auth.ts</code> , <code>accessibility.spec.ts</code>
C2.2.4 — Déploiement progressif & users	4	job <code>deploy</code> Vercel, campagne de tests utilisateurs
C2.3.1  — Cahier de recettes	5	<code>docs/cahier-recettes.md</code> , <code>tests/e2e</code>
C2.3.2 — Correction des bogues	5	<code>docs/plan-correction-bogues.md</code> , <code>CHANGELOG.md</code>
C2.4.1 — Documentation technique	6	<code>README.md</code> , <code>docs/manuel-*.md</code>

1. Introduction et cadrage

► Présentation de MotoTrack et du contexte

MotoTrack est une application web progressive (PWA) de gestion de garage moto : suivi d'entretien, historique kilométrique, rappels d'échéances, recherche de pièces, actualités et météo « pilote ». Le projet n'émane pas d'un client contractuel : il résulte d'une **analyse de marché** que j'ai menée en amont et dont j'ai **déduit un besoin réel**, avant d'endosser moi-même le rôle de commanditaire pour cadrer le produit.

Le constat de départ : l'entretien d'une moto repose encore massivement sur des carnets papier ou des tableurs éparés. Les motards perdent l'historique lors d'une revente, oublient des échéances critiques (chaîne, plaquettes, vidange) et n'ont pas de vue consolidée de leur parc. Les solutions existantes sont soit généralistes (automobile), soit payantes, soit non francophones. Pour ancrer les décisions produit dans un usage concret, j'ai formalisé le besoin autour d'une cible représentative — un club de motards de loisir pris à titre **d'exemple** (« MotoClub Alpes ») : il s'agit d'un *persona d'illustration*, non d'un donneur d'ordre.

PROBLÈME ADRESSÉ

Centraliser, fiabiliser et rendre *portable* l'historique d'entretien d'une ou plusieurs motos, avec des rappels proactifs, sur un outil accessible partout et installable comme une application mobile.

► Objectifs fonctionnels et périmètre de l'application

Objectifs directeurs : fiabilité de la donnée (modèle relationnel plutôt que tableur), mobilité (PWA installable), proactivité (rappels km/périodicité), sécurité & confidentialité (cloisonnement par compte, minimisation RGPD) et accessibilité (produit francophone conforme RGAA).

Domaine	Fonctionnalités livrées (v1.0.0)	Statut
Compte & sécurité	Inscription, connexion, session JWT, protection des routes	Livré
Mon Garage	Ajout de moto (assistant multi-étapes), liste, moto principale	Livré
Carnet d'entretien	Historique des interventions typées, coût, kilométrage	Livré
Catalogue modèles	Suggestion marque/modèle via l'API publique NHTSA vPIC	Livré
Actualités & météo	Flux RSS Google News moto, météo pilote	Livré
Rappels	Modèle de données prêt (seuils km / périodicité)	Socle posé

Périmètre volontairement resserré sur un cœur cohérent et démontrable, dans une logique de produit minimum viable enrichi de façon incrémentale. Cibles : le motard passionné (1 à 3 motos, usage mobile) et le membre de club (sorties de groupe, partage de bons plans).

► Stack technique et justification des choix

Besoin	Choix	Alternative écartée & raison
Framework full-stack	Next.js 14 (App Router)	SPA React pure : pas de SSR/SEO ni de routes API intégrées. Next unifie front, rendu serveur et back.
Langage	TypeScript 5	JavaScript : pas de typage → régressions silencieuses. TS documente les contrats.
Base de données	PostgreSQL 15	NoSQL : domaine fortement relationnel → intégrité référentielle nécessaire.

ORM	Prisma 5	SQL brut : verbeux, non typé. Prisma = client typé + migrations versionnées.
Hébergement	Vercel + Supabase	Cohérence avec Next.js (Vercel) et PostgreSQL managé + pooling (Supabase).
Validation	Zod	Validation manuelle : dispersée, non typée. Zod = source de vérité partagée client/serveur.

REGARD CRITIQUE

Supabase via le *pooler* PgBouncer impose des contraintes et une dépendance à un acteur unique ; le choix est assumé pour la vélocité. La couche d'accès étant isolée derrière Prisma, une migration vers un autre PostgreSQL managé resterait peu coûteuse.

► Méthodologie et organisation du travail

Projet mené en **solo** selon une approche itérative inspirée de Kanban : découpage en incréments fonctionnels, chaque incrément développé sur une branche dédiée, validé par la CI puis fusionné par Pull Request. La discipline de qualité (tests, revue, lint) est portée par l'outillage automatisé — le filet de sécurité d'un développeur unique.

146

tests unitaires

96 %

couverture (statements)

10

routes API

4

entités métier

CAPTURE 1.1

Page d'accueil de MotoTrack (hero « Ton garage. Ton contrôle. » + scène 3D). 1920×1080.

2. Préparation des environnements & CI/CD

C2.1.1 · C2.1.2

Préparer les environnements de déploiement et de test, outiller le suivi qualité/performance, et mettre en place une intégration continue assurant la fusion des sources et le déclenchement de tests réguliers.

► Environnement de développement local (outillage, conteneurisation Docker) C2.1.1

L'environnement local est **reproductible en une commande** grâce à `docker-compose.yml` qui provisionne PostgreSQL 15 et Adminer : tout contributeur obtient une base identique sans installation manuelle. Les variables sensibles sont isolées dans `.env` (exclu du versionnement) et documentées via `.env.example`.

CONFIGURATION PRISMA / SUPABASE

`prisma/schema.prisma` déclare deux URL : `url` (connexion *pooler* PgBouncer, port 6543, à l'exécution en serverless) et `directUrl` (connexion directe, port 5432, pour les migrations) — configuration recommandée pour Prisma sur hébergement serverless.

📸 CAPTURE 2.1

Terminal `docker compose up -d` : conteneurs `mototrack_db` et `mototrack_adminer` démarrés.

► Environnements de test et de déploiement (dev / staging / prod) C2.1.1

Environnement	Rôle	Infrastructure	Données
dev (local)	Développer et tester en isolation	Docker Compose : PostgreSQL + Adminer	Base jetable
staging (préproduction)	Valider chaque PR en conditions réelles	Vercel Preview Deployment (URL éphémère par PR)	Base de test Supabase
prod	Servir les utilisateurs finaux	Vercel (serverless) + Supabase managé	Base de production Supabase

Chaque Pull Request génère un environnement de *staging* automatique (Preview), ce qui permet une validation avant fusion sans mobiliser d'infrastructure dédiée.

📸 CAPTURE 2.2

Tableau de bord Supabase — vue « Database » (tables User, Motorcycle, Maintenance, Reminder générées par Prisma).

► Outils de suivi de performance et de qualité (KPI, linting, monitoring) C2.1.1

Dimension	Outil	Emplacement
Qualité de code	ESLint (<code>next/core-web-vitals</code>)	<code>.eslintrc.json</code> , étape CI « lint »
Typage	TypeScript strict	<code>next build</code> en CI
Sécurité des dépendances	<code>npm audit --audit-level=high</code>	étape CI dédiée (OWASP A06)
Couverture (KPI)	Jest coverage + seuils bloquants	<code>jest.config.js</code>
Disponibilité (KPI)	Sonde de santé + latence DB	<code>src/app/api/health/route.ts</code>
Performance réelle (RUM)	Core Web Vitals (LCP, CLS, INP...)	<code>src/app/api/monitoring/vitals/route.ts</code>

Deux mécanismes de supervision sont implémentés : la **sonde** `/api/health` (exécute un `SELECT 1`, mesure la latence, renvoie un contrat structuré) et le **Real User Monitoring** (`WebVitals.tsx` capte les Core Web Vitals réels et les transmet via `sendBeacon` à `/api/monitoring/vitals`, validés par Zod et journalisés par Winston). Les journaux structurés (Winston, `src/lib/logger.ts`) sont consultables dans les *Runtime Logs* de Vercel.

REGARD CRITIQUE

La CSP est en `Report-Only` le temps de fiabiliser la politique ; une supervision par produit dédié (Sentry/Grafana) est l'étape suivante — l'endpoint Web Vitals est conçu pour l'y brancher.

► **Pipeline d'intégration continue (GitHub Actions : build, tests, lint)** C2.1.2

Le pipeline `.github/workflows/ci.yml` se déclenche sur chaque `push` et `pull_request` vers `main` / `develop`, provisionne un PostgreSQL éphémère et enchaîne des étapes **bloquantes** :

#	Étape	Rôle
1–2	Service <code>postgres:15</code> + <code>npm ci</code>	Base réelle + installation reproductible (lockfile)
3	<code>prisma generate</code> + <code>migrate deploy</code>	Client typé + schéma appliqué
4–5	<code>lint</code> + <code>npm audit</code>	Qualité de code + vulnérabilités (A06)
6	<code>npm run test</code>	146 tests + couverture (seuils bloquants)
7–8	<code>next build</code> + <code>playwright test</code>	Compilation/typage + parcours e2e
9–10	Artefacts + job <code>deploy</code>	Rapports archivés ; déploiement Vercel sur <code>main</code> si tout est vert

 **CAPTURE 2.3**

Exécution GitHub Actions réussie : toutes les étapes vertes jusqu'à `deploy`.

► **Gestion du code source (Git Flow, branches, PR & review)** C2.1.2

Modèle de branches discipliné, adapté au solo mais conforme aux pratiques d'équipe : `main` (production, protégée : fusion uniquement par PR avec CI verte), `develop` (intégration), `feature/*` (un incrément par branche, ex. `feature/i18n-fr`, `feature/monitoring-web-vitals`), `fix/*` (correctifs tracés au plan de bogues). Chaque branche passe par une **Pull Request** décrivant l'intention et la couverture ajoutée ; la PR déclenche CI + Preview ; la fusion exige le succès de l'ensemble. Les messages suivent une convention (`feat:`, `fix:`, `test:`, `docs:`, `ci:`) alimentant le `CHANGELOG.md`.

 **CAPTURE 2.4**

Onglet « Pull requests » + « Branch protection » de `main` (« Require a pull request » et « Require status checks » cochés).

Attendu (C2.1.1 & C2.1.2)	Preuve	Niveau
Environnements automatisés + suivi qualité/perf	Docker, Vercel, Supabase, ESLint, audit, <code>/api/health</code> , RUM	Maîtrisé
Intégration continue + fusion des sources	<code>ci.yml</code> bloquant + branches/PR + protection <code>main</code>	Maîtrisé

3. Conception & architecture

C2.2.1 · éliminatoire

Développer un prototype de l'application en tenant compte de l'ergonomie, des différentes cibles et de la sécurité.

► Architecture logicielle (App Router, couches, modèle de données Prisma) C2.2.1

Couche	Responsabilité	Implémentation
Présentation	Rendu, interactions, états de formulaire	RSC + composants clients (<code>src/app/**/page.tsx</code> , <code>src/components</code>)
Contrôle / API	Contrats HTTP, validation, autorisation	Route Handlers <code>src/app/api/**/route.ts</code>
Services / domaine	Règles transverses réutilisables	<code>src/lib</code> (auth, rateLimit, logger, formatPlate)
Accès aux données	Persistance typée & migrations	Prisma + <code>prisma/schema.prisma</code>
Transversal	Sécurité d'accès aux routes	<code>src/middleware.ts</code>

Le modèle relie quatre entités : `User` 1—N `Motorcycle` 1—N `Maintenance` et `Reminder` , avec suppression en cascade (`onDelete: Cascade`), index sur les clés d'accès et énumération `MaintenanceType` contraignant les types d'intervention côté base.

📁 CAPTURE 3.1

Schéma relationnel de la base (export Supabase ou `prisma-erd`) : `User` → `Motorcycle` → `Maintenance/Reminder`.

► Choix ergonomiques et cibles (web / mobile, PWA) C2.2.1

Cible	Parti pris ergonomique
Desktop	Navigation latérale persistante (<code>AppLayout</code>), grande zone de contenu, scène 3D sur l'accueil.
Mobile	Mise en page responsive (Tailwind), navigation repliée en barre défilable, cibles tactiles agrandies.
PWA	Installable (<code>public/manifest.json</code> + service worker <code>next-pwa</code>), plein écran, icône dédiée.

Interface **intégralement francophone** (cohérente avec la cible et l'accessibilité) ; `<html lang="fr">` déclaré et vérifié par test.

📁 CAPTURE 3.2

Vue mobile responsive (~390px) d'une page authentifiée + installation PWA.

► Sécurité pensée dès la conception (auth JWT, validation SIV) C2.2.1

La sécurité est *structurelle*, pas ajoutée après coup : le **middleware** (`src/middleware.ts`) conditionne l'accès aux routes ; chaque route API **re-vérifie** l'identité (défense en profondeur) et **cloisonne** les données par `userId` ; l'authentification repose sur un **JWT signé (HS256, jose)** en cookie `httpOnly` et des mots de passe **hachés bcrypt**. Les entrées sont validées par **Zod** ; la plaque d'immatriculation est normalisée/validée au format **SIV** (`src/lib/formatPlate.ts`), garantissant une donnée propre dès la saisie.

► Design patterns, best practices et prototype C2.2.1

Patron / pratique	Où & pourquoi
Singleton	<code>src/lib/prisma.ts</code> : instance Prisma unique, évite l'épuisement du pool (rechargements, serverless).
Middleware / chaîne de responsabilité	<code>src/middleware.ts</code> : interception centralisée, vérification JWT, redirection.
Validation en frontière	Zod à l'entrée de chaque route et formulaire (« parse, don't validate »).
DTO / projection	Réponses projetées via <code>select</code> : le <code>passwordHash</code> n'est jamais exposé.
Humble Object	Logique de l'assistant extraite dans <code>garage/new/wizardLogic.ts</code> , testable sans DOM.
Composants à variantes	UI factorisée (<code>src/components/ui</code>) via <i>class-variance-authority</i> + Radix.

Le prototype livré est une application **exécutable de bout en bout** (inscription → ajout de moto → historique), déployable en un push et éprouvée par les tests e2e.



CAPTURE 3.3

Assistant d'ajout de moto (`/garage/new`) : étape de sélection marque/modèle.

Critère C2.2.1	Preuve	Niveau
Architecture & patterns	Couches découplées ; Singleton, middleware, validation frontière, DTO, Humble Object	Maîtrisé
Ergonomie multi-cibles	Responsive + PWA + FR	Maîtrisé
Sécurité dès la conception	Middleware + JWT + bcrypt + validation SIV	Maîtrisé
Prototype fonctionnel	Application exécutable de bout en bout, testée e2e	Maîtrisé

4. Développement des fonctionnalités

C2.2.2 · C2.2.3 · C2.2.4

Développer les fonctionnalités de manière évolutive, sécurisée et accessible ; les doter d'un harnais de tests couvrant l'essentiel du code ; les livrer en continu en vérifiant leur fonctionnement auprès des utilisateurs.

► Implémentation des fonctionnalités clés (Auth, Mon Garage, Carnet d'entretien, Tableau de bord)

Fonctionnalité	Mise en œuvre
Auth	Inscription/connexion Zod + bcrypt, JWT <code>jose</code> en cookie httpOnly, protection par middleware (<code>api/auth/*</code> , <code>auth.ts</code>).
Mon Garage	Assistant multi-étapes (<code>garage/new</code>), CRUD moto cloisonné par utilisateur (<code>api/motorcycles</code>).
Carnet d'entretien	Interventions typées (enum), coût, kilométrage (<code>api/maintenances</code>).
Tableau de bord	Synthèse du parc et des échéances ; intégrations NHTSA vPIC, actualités RSS, météo.

📸 CAPTURE 4.1

Tableau de bord / Mon Garage listant les motos de l'utilisateur connecté.

► Sécurisation du code (OWASP, validation, gestion des secrets)

C2.2.3

Risque OWASP (2021)	Traitement	État
A01 — Contrôle d'accès	Middleware + re-vérification + cloisonnement <code>userId</code>	Traité
A02 — Cryptographie	bcrypt, JWT HS256, cookies <code>secure</code> en prod	Traité
A03 — Injection	Requêtes paramétrées Prisma + Zod	Traité
A05 — Mauvaise configuration	En-têtes + HSTS + CSP (<code>next.config.js</code>)	Traité
A06 — Composants vulnérables	<code>npm audit</code> bloquant en CI	Traité
A07 — Authentification	<i>Rate limiting</i> anti-force brute (429)	Traité
A09 — Journalisation	Winston structuré + sonde de santé	Partiel

Validation : Zod à chaque frontière. **Secrets** : `.env` hors dépôt, `.env.example` documenté, secrets prod dans Vercel, `getJwtSecretKey()` échoue si absent. **Moindre exposition** : le `passwordHash` ne quitte jamais le serveur. Analyse complète : `docs/securite-owasp.md` (annexe).

📸 CAPTURE 4.2

En-têtes de sécurité en production (Network/securityheaders.com) + **réponse 429** après tentatives répétées.

► Accessibilité (RGAA / WCAG)

C2.2.3

Niveau visé : **RGAA 4.1 / WCAG 2.1 AA** sur les parcours critiques, **testé automatiquement** (`tests/e2e/accessibility.spec.ts`).

Critère RGAA	Implémentation	Test
--------------	----------------	------

8.3 — Langue	<code><html lang="fr"></code>	✓ auto
11.1 — Étiquettes	<code><Label htmlFor></code> par champ	✓ auto
11.10 — Erreurs	<code>role="alert"</code> , <code>aria-invalid</code> , <code>aria-describedby</code>	✓ auto
11.13 — Autocomplétion	<code>autocomplete</code>	✓ auto
9.1 — Titres	<code>h1</code> unique	✓ auto
12.7 — Lien d'évitement	« Aller au contenu principal »	manuel

CAPTURE 4.3

Formulaire avec erreurs (FR) + Lighthouse Accessibilité sur `/login` .

► Évolutivité et maintenabilité

- **Typage de bout en bout** : du schéma Prisma aux composants.
- **Découplage** : la logique transverse (`src/lib`) ne dépend ni du rendu ni du transport HTTP.
- **Contrats explicites** : ajouter un type d'entretien = étendre l'énumération + une migration (impact localisé).
- La localisation FR récente a prouvé la capacité du code à évoluer sans casse (tests adaptés, CI verte).

DETTE ASSUMÉE

Le *rate limiting* en mémoire conviendrait à une instance unique ; en serverless multi-instances, un magasin partagé (Redis) serait nécessaire — le contrat de la fonction permet ce remplacement sans toucher aux routes.

► Harnais de tests unitaires et code coverage C2.2.2

Harnais **Jest** + **Testing Library** (unitaire/intégration) et **Playwright** (e2e), selon la *pyramide de tests*.
Priorisation par risque : sécurité/auth d'abord, puis contrats API, logique pure, UI critique. La couverture cible la **couche logique** (déclarée dans `collectCoverageFrom`) et est **verrouillée par des seuils bloquants** (branches 80, fonctions/lignes/instructions 90).

96,06 %

instructions

96,84 %

lignes

98,18 %

fonctions

87,78 %

branches

146 tests / 24 suites, tous verts. Ex. n°1 — `rateLimit.test.ts` vérifie le blocage après quota (anti-force brute) ; ex. n°2 — `api/login.test.ts` verrouille le **message d'erreur générique** quel que soit l'échec (non-énumération des comptes).

CAPTURE 4.4

Sortie `npm run test` : tableau de couverture + « 146 passed / 24 suites ».

► Déploiement continu et progressif + vérification auprès des utilisateurs C2.2.4

Déploiement **continu** (automatisé) et **progressif** : une PR produit un Preview (staging) vérifié avant fusion ; la fusion sur `main` déclenche la production ; « Promote to Production » de Vercel permet un *rollback* immédiat. Une **campagne de test auprès de 5 motards** (préproduction) a piloté des évolutions concrètes :

#	Retour	Action	Suivi
---	--------	--------	-------

R1	Interface par endroits en anglais	Localisation FR des pages compte	Corrigé
R2	Tentatives de connexion illimitées	<i>Rate limiting</i> (429)	Corrigé
R3	Erreurs de formulaire peu visibles	Messages accessibles renforcés	Corrigé
R4	Souhait d'installer sur mobile	Finalisation du manifeste PWA	Corrigé
R5	Saisie du modèle fastidieuse	Suggestion via l'API NHTSA vPIC	Amélioré



CAPTURE 4.5

Vercel — Deployments (Preview + Production) & rapport Playwright vert.

Critères (C2.2.2 / C2.2.3 / C2.2.4)	Preuve	Niveau
Harnais & couverture	146 tests, 96 %, seuils bloquants	Maîtrisé
Sécurité & accessibilité	OWASP traité + RGAA testé	Maîtrisé
Déploiement & vérification utilisateurs	Preview→prod, rollback, campagne 5 testeurs	Maîtrisé

5. Recette des fonctionnalités

C2.3.1 · C2.3.2

Élaborer le cahier de recettes, l'exécuter, analyser les anomalies et bâtir un plan de correction des bogues.

► Cahier de recettes (scénarios de tests, résultats attendus) C2.3.1

Le cahier complet (`docs/cahier-recettes.md`) recense **15 scénarios** reliés aux fonctionnalités et, quand c'est pertinent, à un test automatisé.

ID	Préconditions	Étapes	Résultat attendu	Statut
RT-01	Aucun compte	Inscription valide	Compte créé, redirection connexion	OK
RT-02	—	Inscription e-mail invalide	« Adresse e-mail invalide »	OK
RT-03	—	Mots de passe non concordants	Erreur « ne correspondent pas »	OK
RT-04	Compte existant	Connexion valide	Session ouverte, accès dashboard	OK
RT-05	Compte existant	Mauvais mot de passe	401 générique	OK
RT-06	—	<code>/dashboard</code> sans session	Redirection <code>/login</code>	OK
RT-07	Session	Ajouter une moto	Moto listée	OK
RT-08	Moto existante	Ajouter une intervention	Entrée dans l'historique	OK

► Exécution de la recette et détection des anomalies C2.3.1

ID	Étapes	Résultat attendu	Statut
RT-11	<code>GET /api/health</code>	200 + <code>db: connected</code> + latence	OK
RT-12	Accueil au clavier	Lien d'évitement, focus visible	OK
RT-13	> 5 connexions rapides	429 + <code>Retry-After</code>	OK
RT-14	Langue du document	<code>lang="fr"</code>	OK
RT-15	Installer la PWA	Icône ajoutée, plein écran	OK

Une partie des scénarios est **rejouée automatiquement** par Playwright (`tests/e2e`) : formulaires, protection des routes, contrat de santé, accessibilité — ce qui transforme la recette en filet de non-régression exécuté à chaque CI.

CAPTURE 5.1

Rapport Playwright (`playwright-report`) : tests e2e au vert.

► Analyse des anomalies et régressions

L'exécution manuelle du cahier n'a révélé **aucune anomalie fonctionnelle** : les 15 scénarios passent. Les anomalies traitées relèvent de la **qualité technique** (tests, configuration, accessibilité, sécurité) et ont été détectées par l'outillage (couverture, typage, lint, audit) ou par les retours utilisateurs. Chaque correctif est **accompagné d'un test de non-régression** garantissant la non-réapparition du défaut.

► **Plan de correction des bogues (processus, priorisation, suivi)** C2.3.2

Cycle formalisé : **Détection** → **Qualification (sévérité)** → **Analyse (cause racine)** → **Correction sur branche + test** → **Vérification (CI verte)** → **Clôture (fusion + CHANGELOG)**. Priorisation : Bloquant (immédiat) · Majeur (48 h) · Mineur (itération suivante).

ID	Anomalie	Origine	Correctif	Statut
BUG-01	Couverture insuffisante	Audit qualité	146 tests, seuils relevés	Résolu
BUG-02	Jest exécutait l'e2e	CI	<code>testPathIgnorePatterns</code>	Résolu
BUG-03	Client Prisma périmé	<code>tsc</code>	<code>prisma generate</code>	Résolu
BUG-04	Lint CI bloquant	CI	Ajout <code>.eslintrc.json</code>	Résolu
BUG-05	Formulaires peu accessibles	Audit RGAA + R3	Liaisons ARIA, <code>role="alert"</code>	Résolu
BUG-06	Interface anglophone	Retour R1	Localisation FR + tests	Résolu
BUG-07	Pas de limite de tentatives	Retour R2 + revue	<i>Rate limiting 429</i>	Résolu

Critères (C2.3.1 / C2.3.2)	Preuve	Niveau
Cahier de recettes & exécution	15 scénarios ID/préconditions/étapes/attendu, rejoués en e2e	Maîtrisé
Plan de correction	Processus formalisé + registre tracé + tests de non-régression	Maîtrisé

6. Documentation technique

C2.4.1

Rédiger la documentation technique d'exploitation : réalisation, utilisation, déploiement.

► Stratégie et outils de documentation

Approche « **docs-as-code** » : documentation en Markdown, **versionnée avec le code** et modifiée via les mêmes Pull Requests. Elle évolue au rythme de l'application et reste donc synchrone ; le présent dossier y renvoie plutôt que de la dupliquer.

► Synthèse des manuels (réalisation / utilisation / déploiement)

Document	Public	Emplacement (versions complètes en annexe)
README — vue d'ensemble & démarrage	Développeur	<code>README.md</code>
Manuel de déploiement	Exploitation / DevOps	<code>docs/manuel-deploiement.md</code>
Manuel d'utilisation	Utilisateur final	<code>docs/manuel-utilisation.md</code>
Manuel de mise à jour	Mainteneur	<code>docs/manuel-mise-a-jour.md</code>
Analyses OWASP & RGAA	Technique / QA	<code>docs/securite-owasp.md</code> , <code>docs/accessibilite-rgaa.md</code>

► Traçabilité et support aux évolutions futures

Le `CHANGELOG.md` relie chaque version à son ensemble de changements ; les **migrations Prisma versionnées** tracent l'évolution du schéma ; la version applicative est exposée par `/api/health` . Cet outillage documentaire soutient directement la maintenance et les évolutions futures.

Critère C2.4.1	Preuve	Niveau
Doc de réalisation / utilisation / déploiement	README + 3 manuels + analyses OWASP/RGAA	Maîtrisé

7. Conclusion

► Bilan technique et fonctionnel

MotoTrack est une application full-stack **cohérente, sécurisée et testée**, couvrant les 9 compétences du Bloc 2. Le socle (Next.js, TypeScript, Prisma, PostgreSQL/Supabase) est industrialisé par une CI/CD, un harnais à **96 % de couverture (146 tests)** et une supervision réelle (santé + Web Vitals).

► Difficultés rencontrées et solutions

Difficulté	Solution
Tester des composants React riches sans fragilité	Extraction de la logique en modules purs (<i>Humble Object</i>)
Conflit Jest / Playwright	Séparation des périmètres (<code>testPathIgnorePatterns</code>)
Prisma + serverless (connexions)	Double URL Supabase (pooler + direct) + singleton
Accessibilité difficile à maintenir	Automatisation des vérifications RGAA en e2e
Cohérence de marque & langue	Uniformisation MotoTrack + localisation FR

► Perspectives d'évolution de MotoTrack

- **Rappels actifs** : notifications push (échéances km/temps) via le modèle `Reminder` .
- **CSP bloquante** après période d'observation.
- **Rate limiting distribué** (Redis) pour le serverless multi-instances.
- **Observabilité** : brancher les Web Vitals sur Sentry/Grafana.
- **Internationalisation** au-delà du français.

► Bilan personnel / montée en compétences

Ce projet m'a fait passer d'une logique de « faire fonctionner » à une logique **d'ingénierie** : penser la testabilité *avant* d'écrire, traiter la sécurité comme une propriété structurelle, considérer la CI/CD comme le garant de la qualité d'un développeur solo. J'ai particulièrement progressé sur la conception testable, la sécurité applicative (OWASP, sessions, rate limiting) et l'industrialisation (pipeline, environnements, supervision). Au-delà de l'outil, j'ai appris à **justifier mes choix** et à assumer mes compromis — compétence centrale de l'expert en développement logiciel.

SYNTHÈSE DE CONFORMITÉ — BLOC 2

9 compétences sur 9 démontrées par des preuves du dépôt, dont les 4 éliminatoires sur-argumentées (architecture ch. 3 ; tests, sécurité/accessibilité ch. 4 ; recette ch. 5). Aucune compétence orpheline (voir matrice, sommaire).

8. Annexes

► Annexe A — Documents complets (dans le dépôt)

Les versions longues ne sont pas reproduites ici ; elles sont consultables dans le dépôt aux emplacements suivants :

/README.md	Vue d'ensemble & démarrage
/docs	manuel-deploiement · manuel-utilisation · manuel-mise-a-jour
/docs	securite-owasp · accessibilite-rgaa · cahier-recettes · plan-correction-bogues
/CHANGELOG.md	Journal des versions
/.github/workflows	ci.yml — pipeline CI/CD

► Annexe B — Glossaire technique

App Router	Routage Next.js 14 basé sur le système de fichiers, composants serveur par défaut.
PWA	Progressive Web App : application web installable proche d'une app native.
JWT	JSON Web Token : jeton signé portant l'identité de session.
SIV	Système d'Immatriculation des Véhicules : format des plaques françaises (AA-123-AA).
Pooler (PgBouncer)	Mutualisateur de connexions PostgreSQL, requis en serverless.
CSP / HSTS	En-têtes de sécurité : sources autorisées / forçage HTTPS.
Core Web Vitals / RUM	Métriques de performance perçue mesurées chez les utilisateurs réels.
RGAA / WCAG	Référentiels d'accessibilité (français / international).

► Annexe C — Références & normes

OWASP Top 10 (2021) RGAA 4.1 WCAG 2.1 AA ISO/IEC 25010 RGPD — Outils : Next.js 14, Prisma 5, PostgreSQL 15, Supabase, Vercel, Jest, Playwright, Zod, Winston, Tailwind, Radix.

► Annexe D — Table des captures

Cap. 1.1	Page d'accueil MotoTrack
Cap. 2.1–2.4	Docker · Supabase · run CI · PR & protection de branche
Cap. 3.1–3.3	Schéma BDD · vue mobile/PWA · assistant moto
Cap. 4.1–4.5	Tableau de bord · sécurité/429 · a11y/Lighthouse · tests+coverage · Vercel/Playwright
Cap. 5.1	Rapport Playwright